Internationale Hochschule

IU



Fallstudie zum Thema:

Einsatz von FIDO2 zur Anmeldung auf der Web-präsenz eines Unternehmens

Vorgelegt von:

Ahmed Dib

[redacted]

Matrikelnummer: [redacted]

Abgabedatum: 20.12.2025

Modul: Kryptografische Verfahren

Modulkürzel: [redacted]

Dozent: [redacted]

Inhaltsverzeichnis

Abbildungsverzeichnis III

1. Executive Summary 1

1.1 Zielbild und Nutzen 1

1.2 Entscheidungen (Policy, Fallback, Recovery, Make-or-Buy) 1

2. Ausgangslage und Scope 2

2.1 Kontext & Risiken 2

2.2 Annahmen (Client, Betrieb, Datenschutz) 3

2.3 Scope (In/Out) 3

3. Beteiligte und Verantwortlichkeiten 4

3.1 Wer ist involviert? 4

3.2 Ablaufüberblick (Registrierung/Anmeldung) 4

4. FIDO2-Bausteine im Einsatz 5

4.1 Standards: WebAuthn & CTAP2 5

4.2 Komponenten, die der Betreiber bereitstellen muss 6

5. Anforderungen an die Webanwendung 7

5.1 Voraussetzungen (HTTPS, Domain/RP-ID, Sessions) 7

5.2 Client-Support (Matrix, Legacy, Fallback) 7

5.3 Sicherheitsanforderungen 7

6. Umsetzungsvorschlag 8

6.1 Ziel-Policy (Passkey-first & Step-up) 8

6.2 Nutzer-Flows (Enrollment, Login, Gerätewechsel) 8

6.3 Authenticator-Strategie (Plattform vs. Security Key) 8

6.4 Umgang mit Legacy & Recovery 8

6.5 Make-or-Buy 9

6.6 Risiken & Controls 9

7. Verifikation, Abnahme und Integration von Sicherheit in Projekten 9

7.1 Security im Projekt (Anforderungen, Threats, Gates)..... 9
7.2 Testkonzept (Matrix, Negativtests, Betrieb) 10
7.3 Zertifizierung 10
8. Fazit & Roadmap 10
8.1 Zielerreichung 10
8.2 Restrisiken & nächste Schritte.....11
Literaturverzeichnis.....12

Abbildungsverzeichnis

Abbildung 1: Passkey 1

Abbildung 2: Nutzerfähigkeit 3

Abbildung 3: In-Scope / Out-of-Scope 3

Abbildung 4: Stakeholder 4

Abbildung 5: CTAP & Webauthn 6

1. Executive Summary

1.1 Zielbild und Nutzen

Ziel dieses Vorhabens ist es, die Anmeldung an unserer kostenpflichtigen Webanwendung für mehrere Tausend Kunden so zu modernisieren, dass Phishing- und Kontoübernahme-Risiken messbar sinken und gleichzeitig die Login-Reibung (Passwort-Resets, Supportfälle) reduziert wird. Dafür führen wir FIDO2 in Form eines Passkey-first-Ansatzes ein. Passkeys werden als die primäre Methode zum Anmelden im Produkt verwendet, während es während der Übergangsphase weiterhin eine kontrollierte Sicherheitslösung gibt. Diese Technologie baut auf WebAuthn auf, einer standardisierten Schnittstelle im Browser, die öffentliche Schlüssel zur Authentifizierung nutzt und an den Anbieter oder den Ursprungs-Kontext gebunden ist (W3C, 2021). Es gibt drei Vorteile: (1) ein reduziertes Risiko für wiederholte Angriffe wie Phishing oder Credential Stuffing, da keine Geheimnisse erneut verwendet werden können, (2) weniger Probleme mit Passwörtern, was den Supportbedarf verringert, sowie (3) Kompatibilität durch offene Standards, die keine speziellen Anwendungen erfordern. (W3C, 2021; FIDO Alliance, n. d.).

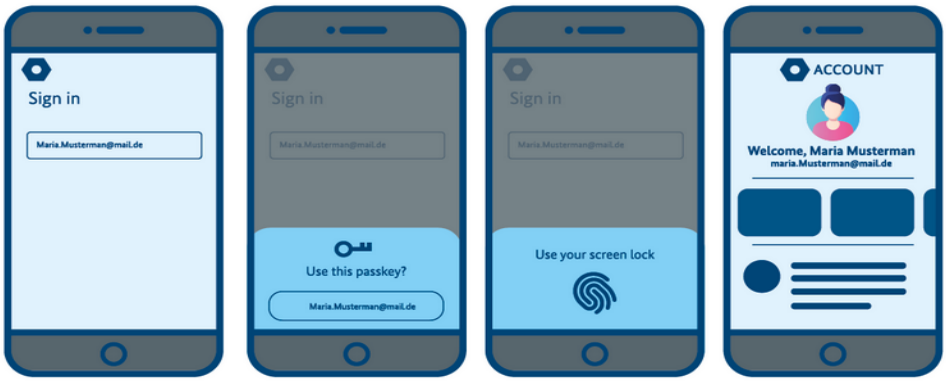


Abbildung 1: Passkey; Quelle: Bundesamt für Sicherheit in der Informationstechnik

1.2 Entscheidungen (Policy, Fallback, Recovery, Make-or-Buy)

Für eine geplante Einführung sind vier wichtige Entscheidungen zu treffen: Erstens die Regelung für die Produkte (erstmal Passkey, mit Übergang zur Alternative und zusätzliche Sicherheit bei wichtigen Aktionen). Zweitens eine Liste, die zeigt, welche Systeme voll unterstützt werden, welche eine Alternative benötigen und welche ein Update erfordern. Drittens ein Plan für die Wiederherstellung und Ausnahmen, der nicht die Sicherheit untergräbt (zum Beispiel durch festgelegte Überprüfungen, Sperrzeiten und

Nachvollziehbarkeit). Viertens die Wahl, ob man selbst verwaltet oder externe Lösungen wie CIAM oder IdP nutzt, zusammen mit einem Prüfplan (positiv und negativ getestet, verschiedene Browser und Geräte, Tests für Missbrauchsfälle; optional Zertifizierung). (FIDO Alliance, n. d.; OWASP, n. d.)

2. Ausgangslage und Scope

2.1 Kontext & Risiken

Die untersuchte Webanwendung ist ein bezahlter Informationsdienst mit tausenden Kunden. Deshalb ist die Anmeldung ein wichtiger Teil des Geschäfts: Wenn Konten missbraucht werden, kann das zu finanziellen Verlusten, Schäden an der Reputation und höheren Supportkosten führen. Zudem wirkt sich die Qualität des Logins unmittelbar auf die Zufriedenheit der Kunden und die Kosten aus, da bei einer großen Anzahl von Nutzern kleine Probleme wie vergessene Passwörter, Kontosperren oder Gerätewechsel schnell zu vielen Supportanfragen werden.

Das Modell, das wir als Ausgangspunkt nehmen, ist das klassische Login mit E-Mail oder Benutzername und Passwort. Dabei kann optional auch eine zweistufige Authentifizierung mit OTP hinzugefügt werden. Im klassischen Modell ist das Passwort ein Geheimnis, das häufig durch Phishing oder Wiederverwendung kompromittiert wird. Das Ziel der Einführung von FIDO2 ist also nicht bloße theoretische Verbesserung, sondern eine praktische Umstellung auf Credentials, die auf Public-Key-Technologie basieren und exakt an die Webanwendung gebunden sind und nur in dem richtigen Kontext genutzt werden können (W3C, 2021).

Ein wichtiger Faktor ist die Unterschiedlichkeit der Kunden: Neben Nutzern mit modernen Smartphones und Laptops gibt es auch solche mit älteren Geräten oder Unternehmensrechnern oder Nutzer, die sich nicht so leicht an neue Anmeldeverfahren gewöhnen. Das ist besonders wichtig für die Planung, denn eine technische Umstellung ohne klare Produkt- und Supportregeln kann zu Problemen beim Login (Verlust von Verkäufen) oder zu unsicheren Lösungen im Support führen. Wie mit Nichtnutzern, alten Systemen und der Wiederherstellung von Konten umgegangen wird, ist daher ein zentraler Teil der Untersuchung.

2.2 Annahmen (Client, Betrieb, Datenschutz)

Es wird angenommen, dass die Webanwendung stabil über HTTPS unter einer konsistenten Domainstruktur betrieben wird (z.B. app.firma.de) und ein zentrales Kontosystem existiert. Die Nutzerbasis ist heterogen. Ein Großteil nutzt aktuelle Browser, ein Teil arbeitet in Legacy-Umgebungen oder möchte keine Passkeys verwenden. Daraus folgt eine definierte Support-Matrix (voll unterstützt / Fallback / nicht unterstützt) als Produktentscheidung. Außerdem wird ein EU-/DSGVO-Kontext angenommen, weshalb Logging und Recovery datenminimiert zu gestalten sind, ohne sicherheitsrelevante Auditierbarkeit zu verlieren (OWASP, n. d.).



Abbildung 2: Nutzerfähigkeit

2.3 Scope (In/Out)

Die folgende Abbildung grenzt den Betrachtungsumfang der Fallstudie (In-Scope) von bewusst nicht behandelten Themen (Out-of-Scope) ab.

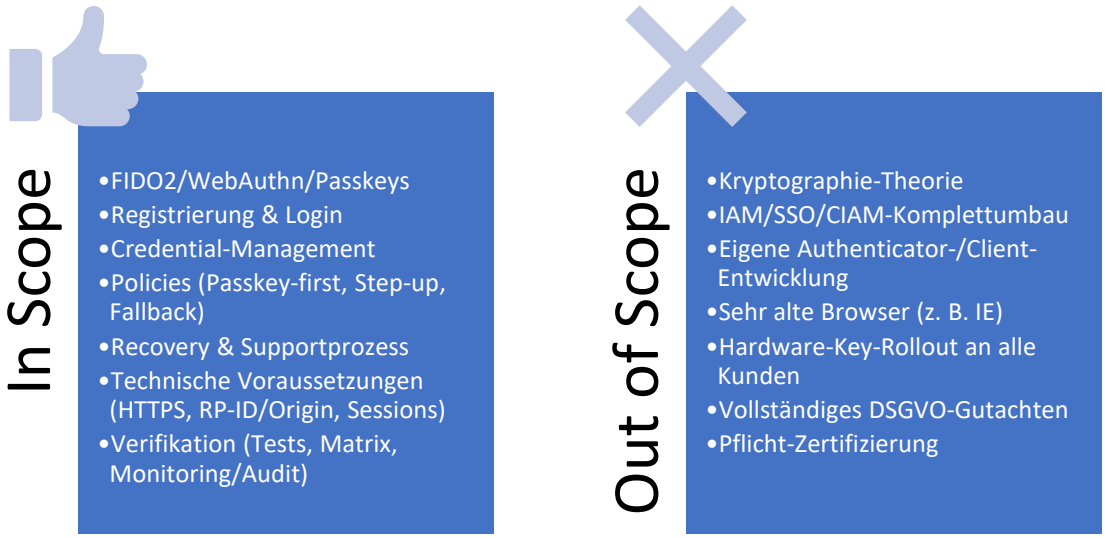


Abbildung 3: In-Scope / Out-of-Scope

3. Beteiligte und Verantwortlichkeiten

3.1 Wer ist involviert?

Abbildung 4 zeigt die am FIDO2-Login beteiligten Stakeholder:

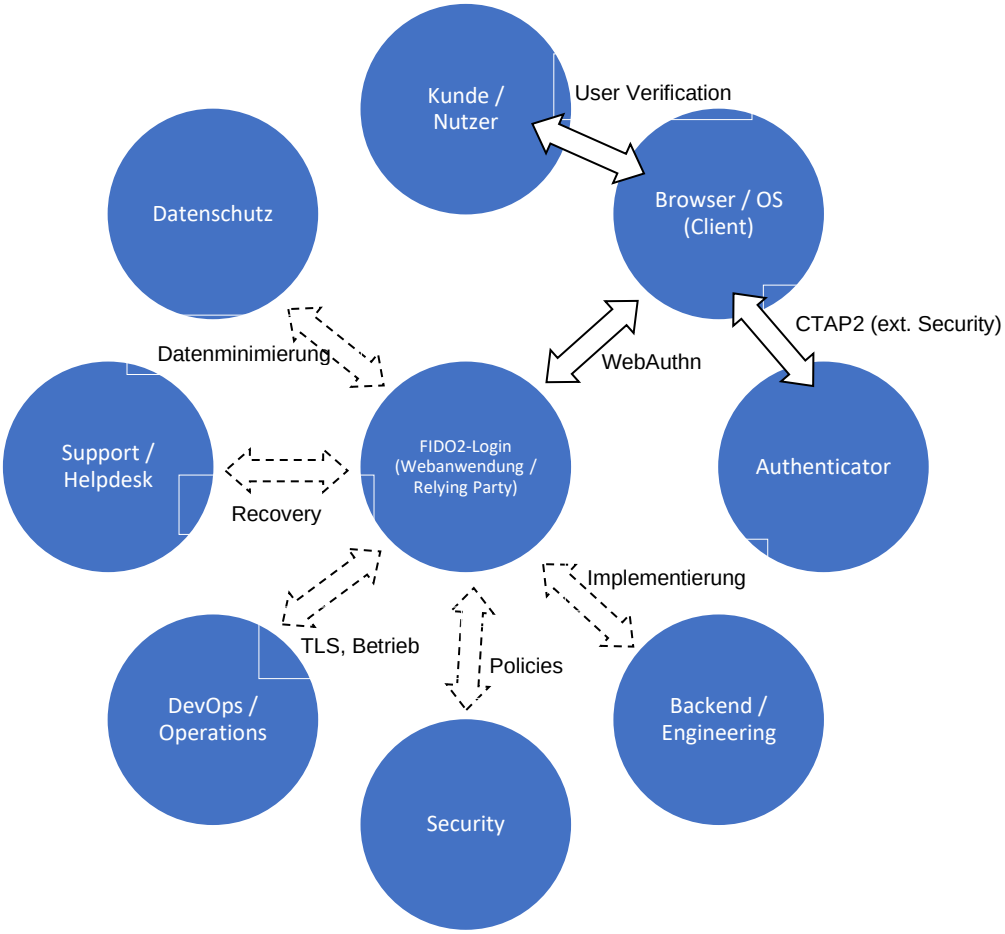


Abbildung 4: Stakeholder

3.2 Ablaufüberblick (Registrierung/Anmeldung)

Tabelle 1: FIDO2 Ablauf - Registrierung (Passkey erstellen)

Schritt	Nutzer	Browser/OS (Client)	Authentica-tor (Passkey, Security Key)	Webanwendung (Relying Party / Server)
Nutzer startet Registrierung	Klickt Passkey hinzu-fügen	-	-	Startet WebAuthn Registration
Server er-zeugt Chal-lenge	-	Empfängt PublicKeyCredentialCre-ationOptions (Challenge, RP-ID, User-ID, Policy)	-	Generiert Challenge, sendet Optionen an Browser

Authenticator erzeugt Keypair	Bestätigt lokal (Biometrie/PIN/Touch)	Ruft WebAuthn navigator.credentials.create() auf	Erzeugt Schlüsselpaar, speichert privaten Schlüssel sicher; erstellt Antwort	-
Server speichert Credential	-	Sendet credential (Credential ID, Public Key, Signaturdaten/Attestation) an Server	-	Prüft Antwort (Challenge/Origin/RP-ID), speichert Public Key + Credential ID im Konto

Tabelle 2: FIDO2 Ablauf – Anmeldung (Authentifizieren)

Schritt	Nutzer	Browser/OS (Client)	Authenticator (Passkey, Security Key)	Webanwendung (Relying Party / Server)
Nutzer startet Login	Klickt 'Mit Passkey anmelden'	-	-	Startet WebAuthn Authentication
Server erzeugt Challenge	-	„Empfängt PublicKeyCredentialRequestOptions (Challenge, allowCredentials optional, Policy)	-	Generiert Challenge, sendet Optionen an Browser
Authenticator signiert Challenge	Bestätigt lokal (Biometrie/PIN/Touch)	Ruft navigator.credentials.get() auf	Wählt passendes Credential, signiert Challenge mit **privatem Schlüssel**	-
Server verifiziert Signatur	-	Sendet Assertion an Server (Credential ID, Signatur, ClientDataJSON, AuthenticatorData)	-	Prüft Challenge/Origin/RP-ID, verifiziert Signatur mit **Public Key**, setzt Session

4. FIDO2-Bausteine im Einsatz

4.1 Standards: WebAuthn & CTAP2

Für die Nutzung von FIDO2 sind zwei Standards wichtig: WebAuthn und CTAP2. WebAuthn ist eine Schnittstelle im Browser, über die unsere Webanwendung als Vertrauenswürdige Partei (Relying Party) starke Anmeldedaten auf der Grundlage öffentlicher Schlüssel registriert und zur Anmeldung verwendet. Wichtig dabei ist, dass ein

registrierter Anmeldeantrag (credentials) an die Vertrauenswürdige Partei oder an den Ursprungskontext gebunden ist und daher nicht einfach auf anderen Webseiten wiederverwendet werden kann (W3C, 2021). CTAP2 beschreibt, wie der Browser oder das Betriebssystem mit externen Sicherheitschlüsseln (z. B. über USB, NFC oder Bluetooth) kommuniziert. Für uns ist besonders wichtig, welche Arten von Sicherheits Schlüsseln (Authenticator-Typen) wir unterstützen können (FIDO Alliance, 2018). In der Kommunikation mit den Nutzern wird häufig von Passkeys gesprochen, da diese die praktische Umsetzung von FIDO2 für Endverbraucher darstellen und eine Anmeldung ohne Passwort über Schlüsselpaare ermöglichen (FIDO Alliance, o. D.).

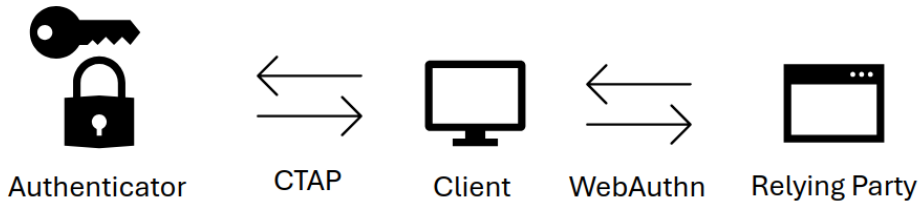


Abbildung 5: CTAP & Webauthn

4.2 Komponenten, die der Betreiber bereitstellen muss

Betreiberseitig ist eine belastbare Relying-Party-Implementierung erforderlich. Diese muss Registrierungs- und Login-Vorgänge serverseitig korrekt steuern, insbesondere durch sichere Challenge-Erzeugung und die Validierung der Rückgaben aus dem WebAuthn-Client (W3C, 2021). Die Anwendung benötigt zudem ein Datendokument, das für jeden Benutzer mindestens die Credential-ID, den Public Key und die notwendigen Daten verwaltet und mehrere Credentials pro Benutzer erlaubt. Dabei werden die privaten Schlüssel nicht auf dem Server gespeichert. Der Server speichert nur die Informationen, die für die Überprüfung benötigt werden. Es müssen zudem Regeln festgelegt und eingehalten werden, beispielsweise zur lokalen Bestätigung durch den Benutzer, der Einsatz externer Nachweisgeräte sowie die Strategie zur Prüfung der Authentizität (z. B. optional für gewöhnliche Benutzer und strenger für Administrator-Konten). Zuletzt ist es notwendig, Protokolle und Überwachung zu betreiben, um Fehler beim Anmelden oder Registrieren, auffällige Muster und wichtige Ereignisse für die Wiederherstellung nachvollziehbar zu machen, ohne dabei zu viel personenbezogene Daten zu sammeln.

5. Anforderungen an die Webanwendung

5.1 Voraussetzungen (HTTPS, Domain/RP-ID, Sessions)

WebAuthn- oder Passkeys funktionieren im Web nur zuverlässig, wenn die Anwendung in einem sicheren Umfeld läuft, also die entsprechenden Seiten über HTTPS geladen werden (Mozilla, 2025). Darüber hinaus ist ein klares System für Domains und Ursprünge nötig, denn WebAuthn-Zertifikate werden an die Anbieterseite und den Ursprungskontext gebunden (W3C, 2021). Zudem sollten grundlegende Sicherheitsmaßnahmen weiterhin angewandt werden, besonders das Verwalten von Sitzungen: Nach einer erfolgreichen Anmeldung sollte die Session-ID aktualisiert werden, um das Risiko von Sitzungsfälschung zu verringern (OWASP, n. d.).

5.2 Client-Support (Matrix, Legacy, Fallback)

WebAuthn erfordert keinen Spezialbrowser, aber einen modernen Browser-/Plattform-Stack. Sehr alte Clients (z. B. Internet Explorer) sind nicht kompatibel (Fyrd, 2025). Daraus folgt eine verbindliche Support-Matrix mit drei Klassen: „voll unterstützt“ (Passkeys/WebAuthn), „eingeschränkt“ (Fallback) und „nicht unterstützt“ (Upgrade erforderlich). Zusätzlich wird clientseitig eine Feature-Erkennung umgesetzt, damit Passkey-Funktionen nur dann angeboten werden, wenn WebAuthn im jeweiligen Kontext verfügbar ist (Mozilla, 2025).

5.3 Sicherheitsanforderungen

Auf der Serverseite müssen WebAuthn-Zeremonien so umgesetzt werden, dass bei jedem Vorgang eine neue, frische Challenge erzeugt wird und diese zuverlässig beim Abschluss des Vorgangs überprüft wird. MDN weist darauf hin, dass zufällige Challenges notwendig sind und beschreibt die üblichen Prüfungen zur Validierung (Mozilla, 2025). Zusätzlich müssen Signaturen mit dem gespeicherten Public Key verifiziert und Kontextprüfungen (Challenge, RP/Origin) korrekt durchgeführt werden (W3C, 2021). Da Login-, Recovery- und Credential-Management-Endpunkte typische Ziele automatisierter Angriffe sind, werden Rate-Limits, Brute-Force-Schutz und Monitoring verpflichtend eingeführt auch für Forgot-Password-Flows, da diese wie Login-Endpunkte abgesichert werden sollen (OWASP, n. d.; OWASP, 2023). Diese Anforderungen werden als verbindliche Abnahmekriterien getestet (Challenge/Origin/RP-ID, Rate-Limits, Recovery-Härtung, Session-Handling) (OWASP, n. d.; W3C, 2021).

6. Umsetzungsvorschlag

6.1 Ziel-Policy (Passkey-first & Step-up)

Empfohlen wird Passkey-first mit Übergangsphase: Passkeys sind die bevorzugte Login-Methode, ein Passwortpfad bleibt zunächst als Fallback für nicht WebAuthn-fähige oder ablehnende Nutzer bestehen. Für wichtige Sicherheitsaktionen wie das Ändern von Kontodaten oder Zahlungsinformationen, das Exportieren sensibler Daten oder das Verwalten von Anmeldeinformationen muss ein zusätzlicher Sicherheitsschritt mit Passkey oder WebAuthn durchgeführt werden. Passkeys sind FIDO-Credentials für passwortlose Authentifizierung und ersetzen Passwörter durch kryptografische Schlüsselpaare. Phishing-resistente Verfahren sollen aktiv gefördert werden (FIDO Alliance, o. D.; NIST, 2025).

6.2 Nutzer-Flows (Enrollment, Login, Gerätewechsel)

Passkey-Enrollment wird nach erfolgreichem Login sowie im Profil („Sicherheit“) angeboten, jedoch nur bei frischer Authentisierung/Step-up, um das unautorisierte Hinzufügen neuer Credentials zu verhindern. Der Login ist UI-seitig klar („Mit Passkey“ primär, „Mit Passwort“ sekundär). Mehrere Credentials pro Konto werden unterstützt, Passkeys sind benennbar und widerrufbar. Nutzer werden früh zu einem Backup-Credential geführt (FIDO Alliance, o. D.).

6.3 Authenticator-Strategie (Plattform vs. Security Key)

Unterstützt werden Plattform-Passkeys (geringste Reibung) und Security Keys als Alternative. Für Step-up und risikoreiche Vorgänge wird User Verification required gesetzt (lokale Bestätigung per PIN/Biometrie). Externe Security Keys nutzen CTAP2 zwischen Client/Plattform und roaming Authenticator (FIDO Alliance, 2018).

6.4 Umgang mit Legacy & Recovery

Der Parallelbetrieb ist bewusst, aber der Fallback darf nicht zum „Default“ werden. Passkeys werden prominent angeboten und per Feature-Erkennung nur bei WebAuthn-Verfügbarkeit angezeigt. Recovery wird wie ein Login-Endpunkt abgesichert (Rate-Limits, Brute-Force-Schutz, kurzlebige Tokens) und Supportfälle benötigen definierte Identitätsprüfung, Sperrfristen und Auditierbarkeit (OWASP, n. d.; OWASP, 2023).

6.5 Make-or-Buy

Für die Umsetzung wird ein CIAM/IdP-Ansatz bevorzugt, sofern keine bereits vorhandene interne Identitätsplattform mit WebAuthn-Unterstützung existiert. Das liegt daran, dass Enrollment, Gerätewechsel, Recovery und Auditierbarkeit die größten Betriebsaufwände verursachen und durch Funktionen von Anbietern schneller und stabiler abgedeckt werden können (Auth0, n. d.; Okta, n. d.; Microsoft, 2025). Eine Eigenentwicklung bleibt sinnvoll, wenn regulatorische Vorgaben Auslagerung unattraktiv machen oder die Authentifizierung strategisch intern betrieben werden soll. In diesem Fall müssen zudem zusätzliche Aufwendungen für Tests, Abuse-Cases und Supportprozesse berücksichtigt werden.

6.6 Risiken & Controls

Die größten Risiken sind ein Downgrade auf Passwort oder Recovery sowie der Missbrauch von Prozessen. Dagegen wirken Step-up für kritische Aktionen, das Überwachen von Änderungen an Recovery und Passwörtern sowie das „Passkey hinzufügen“ nur bei einer frischen Authentifizierung. Sicherheit der Session bleibt wichtig (erneuerung der Session-ID nach Anmeldung) und Domain, Origin und RP-ID werden früh festgelegt, da WebAuthn an diesen Kontext gebunden ist (OWASP, n. d.; W3C, 2021).

7. Verifikation, Abnahme und Integration von Sicherheit in Projekten

7.1 Security im Projekt (Anforderungen, Threats, Gates)

Damit FIDO2 nicht nur eingebaut, sondern auch richtig sicher genutzt wird, werden zu Beginn des Projekts klare Sicherheits-Anforderungen festgelegt, die überprüft werden können (zum Beispiel: Passkey-Login verfügbar, Fallback mit Rate-Limit, Step-up bei wichtigen Aktionen, Recovery nicht weniger sicher als das Login). Dieses Vorgehen folgt dem Prinzip, Sicherheit von Anfang an in den Entwicklungsprozess einzubinden und nicht erst am Ende durch Tests zu prüfen (NIST, 2022). Auf diese Weise wird ein kleineres, aber genaues Sicherheitsmodell für die neuen Authentifizierung und Recovery-Flows erstellt. In diesem Fall werden Risiken wie Downgrade auf Passwort/Recovery, Social Engineering im Support, Session-Hijacking und Fehler bei der Prüfung von Challenge und Origin besonders beachtet. Aus diesen Risiken werden Sicherheitsmaßnahmen wie Step-up, Rate-Limits/Monitoring, Regeneration von Session-IDs und Audit-Logs bei Änderungen der Zugangsdaten abgeleitet. Diese Maßnahmen sind Voraussetzungen für die Freigabe eines Releases. Änderungen an den Login- und Recovery-Flows benötigen zwingend eine Sicherheits-Prüfung und Nachweise (NIST, 2022).

7.2 Testkonzept (Matrix, Negativtests, Betrieb)

Die Prüfung wird in drei Schritten durchgeführt. Zunächst werden die wichtigsten Nutzerpfade end-to-end getestet: die Registrierung und Anmeldung mit Passkeys, die Verwaltung mehrerer Anmeldeinformationen (einschließlich Widerruf und Löschen), der Wechsel des Geräts sowie der Fallback-Login. Diese Tests werden an eine festgelegte Auswahl an Browsern und Geräten durchgeführt, um die Kompatibilität klar nachzuweisen. Als Grundlage dient die WebAuthn-Spezifikation (W3C, 2021). Im zweiten Schritt werden gezielte Tests durchgeführt, um typische Fehler beim Aufbau zu erkennen: Wiederholte Anfragen (Challenge-Replay), falscher Ursprung- (Origin) oder falscher RP-Context, unerlaubte Wiederverwendung von Assertions, das Anbinden von Anmeldeinformationen an ein anderes Konto sowie die Umgehung von Step-Up-Regeln. Für die Struktur und den Umfang dieser Tests können die Standards von OWASP (ASVS, WSTG) genutzt werden (OWASP, n. d.). Drittens wird die Betriebswirksamkeit geprüft: aktive Rate-Limits (Login/Recovery/Credential-Management), Alarmierung bei auffälligen Mustern, Auditierbarkeit von Credential- und Recovery-Ereignissen sowie eine Auswertung, ob der Rollout Supportlast reduziert oder verlagert (OWASP, n. d.).

7.3 Zertifizierung

Eine formale FIDO-Zertifizierung ist für die vorliegende B2C-Einführung optional. Verpflichtend ist jedoch eine harte interne Verifikation (Negativtests, Matrix, Logging/Monitoring, Abuse-Cases) als Release-Gate (OWASP, n. d.). Zertifizierung wird nur empfohlen, wenn sie extern gefordert wird oder als Vertrauensnachweis strategisch genutzt werden soll (FIDO Alliance, n. d.).

8. Fazit & Roadmap

8.1 Zielerreichung

Das Konzept erfüllt das Ziel, die Anmeldung für eine große Kundenbasis spürbar sicherer und gleichzeitig nutzerfreundlich zu gestalten. Durch FIDO2/WebAuthn mit Passkey-first wird die Anmeldung phishingschwieriger, weil die Bestätigung lokal am Gerät erfolgt und der Server die kryptografische Antwort im Kontext von Relying Party/Origin verifiziert. Zugleich bleibt die Lösung praxistauglich, da sie als gestufter Rollout (Passkeys bevorzugt, Fallback kontrolliert) geplant ist und die entscheidenden Betriebsfragen (Legacy, Recovery, Step-up, Make-or-Buy, Verifikation) als konkrete Entscheidungen für das technische Gremium abbildet.

8.2 Restrisiken & nächste Schritte

Restrisiken bestehen vor allem in Legacy-Clients (Kompatibilität), Downgrade auf Fallback/Recovery sowie Social-Engineering im Support. Die Roadmap folgt daher drei Phasen: (1) Passkeys produktiv mit Support-Matrix und abgesichertem Recovery ausrollen, (2) Adoption erhöhen und Step-up für kritische Aktionen konsequent erzwingen, (3) Passwortpfad abhängig von Nutzung und Risiko schrittweise zurückbauen; Zertifizierung/MDS nur für High-Assurance-Konten optional prüfen (FIDO Alliance, n. d.; OWASP, n. d.).

Literaturverzeichnis

1. Auth0. (n. d.). *FIDO authentication with WebAuthn*. https://auth0.com/docs/secure/multi-factor-authentication/fido-authentication-with-webauthn
2. Auth0. (n. d.). *Passkeys*. https://auth0.com/docs/authenticate/database-connections/passkeys
3. FIDO Alliance. (2018, February 27). *Client to Authenticator Protocol (CTAP) (Version 2.0)*. https://fidoalliance.org/specs/fido-v2.0-id-20180227/fido-client-to-authenticator-protocol-v2.0-id-20180227.html
4. FIDO Alliance. (2025, July 14). *Client to Authenticator Protocol (CTAP) (Version 2.2)*. https://fidoalliance.org/specs/fido-v2.2-ps-20250714/fido-client-to-authenticator-protocol-v2.2-ps-20250714.html
5. FIDO Alliance. (2025, May 21). *FIDO Metadata Service (MDS) v3.1 (Specification)*. https://fidoalliance.org/specs/mds/fido-metadata-service-v3.1-ps-20250521.html
6. FIDO Alliance. (n. d.). *Conformance self-validation testing*. https://fidoalliance.org/certification/conformance/
7. FIDO Alliance. (n. d.). *FIDO Functional Certification Program*. https://fidoalliance.org/certification/functional-certification/
8. FIDO Alliance. (n. d.). *FIDO Metadata Service (MDS) overview*. https://fidoalliance.org/metadata/
9. FIDO Alliance. (n. d.). *Functional certification: Servers*. https://fidoalliance.org/certification/functional-certification/functional-certification-servers/
10. FIDO Alliance. (n. d.). *Passkeys: Passwordless authentication*. https://fidoalliance.org/passkeys/
11. FIDO Alliance. (n. d.). *User authentication specifications overview*. https://fidoalliance.org/specifications/
12. Fyrd. (2025). *Web Authentication API – Browser support*. Can I use…. https://caniuse.com/webauthn

13. Microsoft. (2025, November 6). *Support for FIDO2 authentication with Microsoft Entra ID*. https://learn.microsoft.com/en-us/entra/identity/authentication/concept-fido2-compatibility

14. Microsoft. (2025, November 18). *Enable passkeys (FIDO2) for your organization*. https://learn.microsoft.com/en-us/entra/identity/authentication/how-to-enable-passkey-fido2

15. Microsoft. (2025, November 18). *Passkeys (FIDO2) authentication method in Microsoft Entra ID*. https://learn.microsoft.com/en-us/entra/identity/authentication/concept-authentication-passkeys-fido2

16. Mozilla. (2025, October 9). *Web Authentication API (WebAuthn).* MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Web_Authentication_API

17. National Institute of Standards and Technology. (2022, February). *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities (SP 800-218).* https://csrc.nist.gov/publications/detail/sp/800-218/final

18. National Institute of Standards and Technology. (2025). *NIST Special Publication 800-63B: Authentication and lifecycle management.* https://pages.nist.gov/800-63-4/sp800-63b.html

19. Okta. (n. d.). *Configure the FIDO2 (WebAuthn) authenticator*. https://help.okta.com/oie/en-us/content/topics/identity-engine/authenticators/configure-webauthn.htm

20. Okta Developer. (n. d.). *Web Authentication integration guide*. https://developer.okta.com/docs/guides/authenticators-web-authn/aspnet/main/

21. OWASP. (2023). *API Security Top 10 – Broken Authentication (0xa2).* https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/

22. OWASP. (n. d.). *Application Security Verification Standard (ASVS).* https://owasp.org/www-project-application-security-verification-standard/

23. OWASP. (n. d.). *Forgot Password Cheat Sheet.* OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html

24. OWASP. (n. d.). *Session Management Cheat Sheet.* OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

25. OWASP. (n. d.). *Testing Multi-Factor Authentication.* Web Security Testing Guide. https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/06-Authentication_Testing/09-Testing_for_Strong_Cryptography

26. OWASP. (n. d.). *Web Security Testing Guide (WSTG).* https://owasp.org/www-project-web-security-testing-guide/

27. World Wide Web Consortium. (2021, April 8). *Web Authentication: An API for accessing Public Key Credentials (Level 2) (W3C Recommendation).* https://www.w3.org/TR/webauthn-2/

28. World Wide Web Consortium. (2025, January 27). *Web Authentication: An API for accessing Public Key Credentials (Level 3).* https://www.w3.org/TR/webauthn-3/